

Project Title: Detecting Pitch Correction in Popular Songs

Group Members: Mark Ma, Justin Peng, Sarah Wilson

1. Problem:

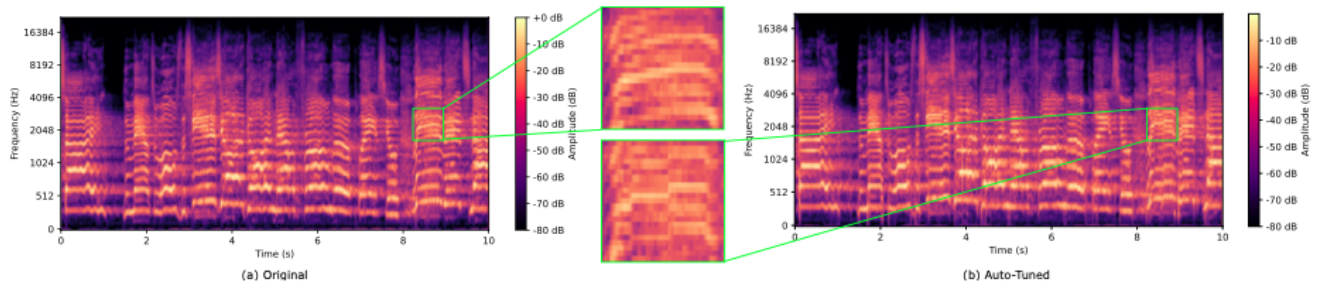
In modern music production, pitch correction software such as Auto-Tune and Melodyne are widely used to correct vocal performances. While this technology is often applied subtly, it can sometimes be used excessively, raising questions about the authenticity of live performances and studio recordings. This project aims to develop a method to detect and visualize pitch correction in popular songs. Our approach will allow us to analyze whether a song has undergone pitch correction, similar to analyses by content creators like Wings of Pegasus on YouTube. This project may have applications in identifying live lip-syncing performances, assessing production techniques, and examining trends in music production. Furthermore, we want to use our project to run a comprehensive analysis on songs across different time periods and cultures from which we can draw some socio-cultural insights about how technology has revolutionized the music industry.

2. Academic Research & Preparation:

Pitch correction, as its name suggests, modifies the raw vocal audio by correcting off-pitch notes to create a polished product. At a high level, there are two types of pitch correction: subtle correction and extreme correction. Subtle corrections are commonly used in an inauthentic way to correct the singer's voice to a perfect note. Extreme correction, however, is usually used intentionally to achieve a particular sound effect, which is widely used in modern pop and hip-hop music. As mentioned in the objective of this project, we aim to research a methodology for detecting subtle correction, which has sparked numerous debates in the history of music. However, since modern pitch correction software has evolved through many iterations, the algorithm has made the correction subtle enough that the human eye—or even detection software—can hardly detect it. This brings us to the challenge of this project, and we might need to conduct empirical and observational studies, including manually analyzing the waveform.

Before we initialize our research, we first have to understand the theoretical background behind the pitch correction software. The pitch correction process breaks down into 3 major components: pitch tracking, pitch modification and resynthesis. In pitch tracking, the software analyzes the singer's pitch by tracking the fundamental frequency of the vocal signal. The detected pitch is then compared to a predefined scale. The next step is pitch modification. For an off-pitched note, pitch modification is applied to adjust the pitch to the nearest correct music note. This process can be instant (hard tuning) or gradual (natural tuning). The quality of the pitch-correction software depends on this step since a good software can preserve almost perfect original voice timbre while an average software will cause some subtle inconsistencies and unnatural sound formation. The final step is resynthesis. In this step, the corrected sound will be merged back into the original track. Similarly, this step is also a determining factor for the quality of a software. With an advanced algorithm, the modified sound can blend in the original sound seamlessly by maintaining the natural vibrato.

The research also outlined a few approaches for detecting pitch correction. The most common one mentioned is spectrogram analysis. Usually, the pitch-corrected waveforms exhibit a different fluctuation pattern. As illustrated in the two diagrams below, the left represents the original sound, whereas the right represents the auto-tuned sound. After zooming in, we can see a prominent difference between the original and auto-tuned waveforms. In the original sound, the transition is smooth and gradual. In the auto-tuned waveform, however, we observe this ‘step’ pattern, which is mechanical and impossible to achieve without manipulation.



The other method of analysis is to compare live performances and studio recordings of the same artist singing the same song. In live performances, the extent to which the sound is corrected is minimized because real-time manipulation in live performances is much harder than in a studio. As such, we expect to see discrepancies.

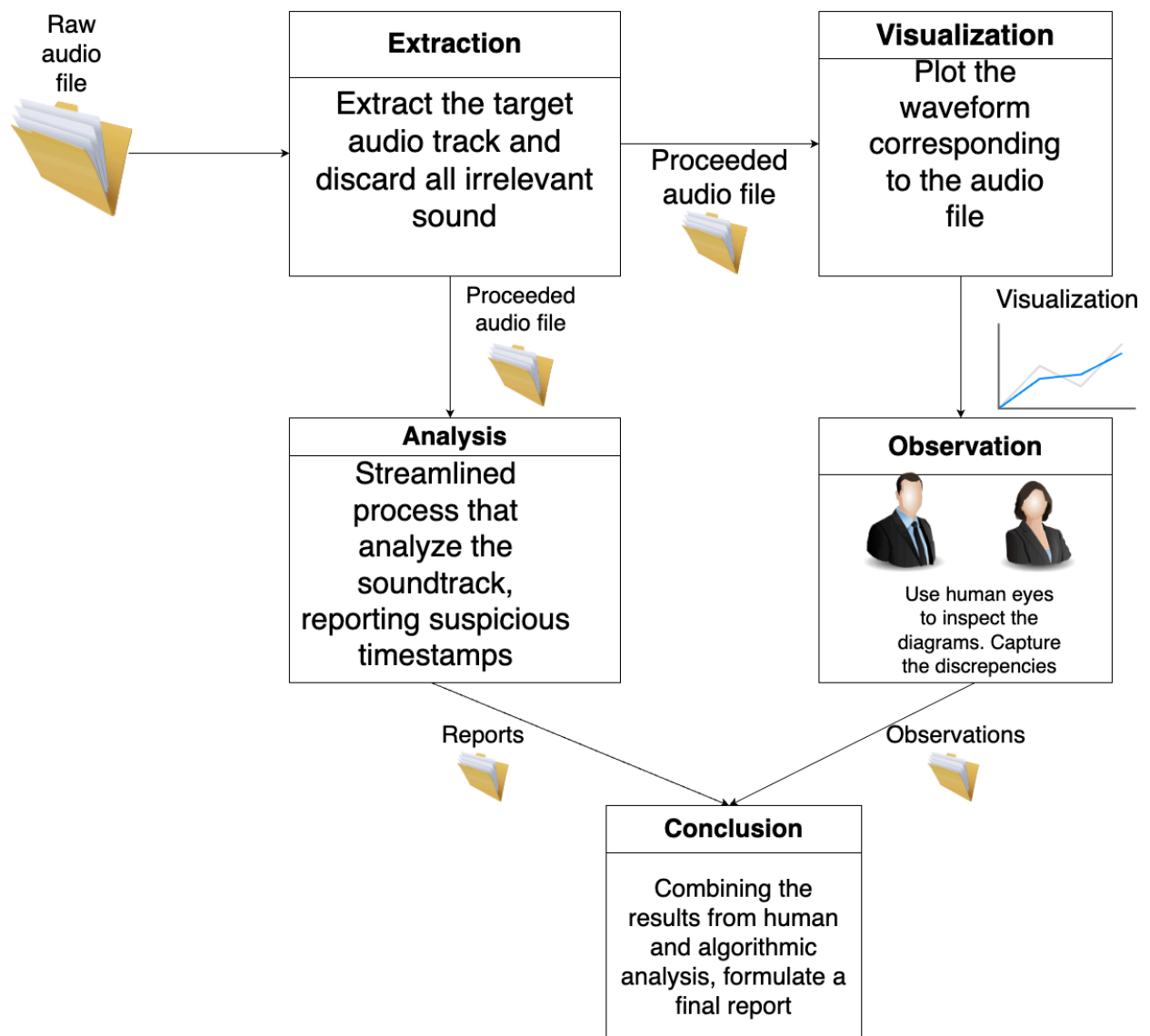
The research also highlighted potential challenges and limitations. Firstly, the subtle corrections are hard to capture. Given the two diagrams above, the discrepancies happened at a very local level, to the point where the human eye is unable to catch them without zooming in closely. Hence, our research needs to process each short timestamp range one by one to produce effective results. Another challenge is that the real-life recordings are oftentimes not ideal for our research. For instance, there are usually a lot of background noises (fans, other harmony singers) interfering with the quality of the raw audio. To tackle that, we first have to pre-process the raw audio files to extract the target main audio track.

3. Methods:

The process is broken down into multiple modules as shown in the figure below (Fig. 1). The modules are extraction, visualization, analysis, observation and conclusion. Note that, to better implement the pipeline, we ensured logical independence, which means that each module is working independently and has no knowledge of other modules. Our design not only reduced dependency overhead but also simplified the process of assigning work internally between our team members. Here is a detailed explanation of our approach.

We start the process by collecting raw files online. The raw files encompass all vocal performances including but not limited to: concerts, albums, online singers' performances, and busk singers. We will search from a variety of sources such as Youtube and Spotify. Meanwhile, we will also do cross-culture experiments by collecting data points from other countries/languages and comparing the data we find here in Western

culture. We want to validate the accuracy and stability of the research model under various linguistic contexts. The collection of the raw data files is most likely to be done through an exhaustive manual search. However, the stretch goal of the project is to create a web-scraping Python script that can automate this process. In this way, we can largely increase the sample size, which will result in a much more accurate statistical inference.



(Fig. 1)

3.1 Extraction

Our next module is Extraction. In this module, our main objective is to extract the target vocal audio and filter out interfering sounds such as fans' noise, harmony sounds, instruments and so on. This process is substantially complicated and involves multiple stages such as signal processing techniques, deep learning models, and audio separation

methodologies. Before our formal research, our team already proposed several potential challenges in this extraction process. First of all, there might be some difficulties in differentiating the lead vocals and background harmonies—both of which are human voices with similar timbre. Although there are numerous potential methods to tackle this issue, we are still sceptical about how these algorithms can work in real-life scenarios instead of in the sample tests with ideal settings. Secondly, we also need to take into account how much and how severe the background noises are. Street busking, in particular, might have higher uncertainties due to equipment limitations. For example, how do we handle the case when the speaker is making unexpected noises? How do we handle the case when the performer is interacting with the audience and pauses the performance? Our algorithm needs to handle these cases gracefully, or at least not significantly undermine the results of the research.

Despite these concerns above, there are a few existing approaches to this issue. There are few traditional approaches that are still commonly used today. But all of them have more or less some deficiencies. For instance, Spectral Subtraction can effectively eliminate stationary noises by identifying the constant frequency of a particular noise, for example, a constant vibration caused by a machine. However, this approach does not handle unexpected and abrupt noises. Another traditional approach is Independent Component Analysis, also known as ICA. In this approach, we take each source independently and try to separate the sources into independent signals. However, this approach also has limitations as the inability to distinguish overlapping frequencies. More specifically, it cannot distinguish between human sound and the background music which have the same pitch. Due to the above deficiencies, we decided to use some more advanced ML-based approaches which significantly improved the accuracy of vocal separation tasks. There are a few approaches that we consider as our candidates. First, Deep U-Net Architectures (Spleeter, Demucs, Open-Unmix). This architecture uses convolutional neural networks to separate vocals from music. It is pre-trained on large datasets of mixed and isolated tracks. This model works extremely well and is stable for studio recordings. Second, we have Recurrent Neural Networks (RNNs) & Transformers. RNNs model temporal dependencies in the vocal track. Meanwhile, the transformers can model long-range relationships in an audio stream. Overall, this model can effectively solve the “overlapping frequency” issue, so the background music can be removed. Third, self-supervised learning. This model trains on raw, unlabeled data to improve robustness. In other words, this can effectively resolve the issue we mentioned before that live performances frequently have unexpected and unpredictable sounds such as clapping hands, cheering, audience singing alone and so on.

As far, we have a basic theoretical understanding of how extraction works. Given in the research, we noticed that there is no all-in-one approach that can handle all circumstances. In our project, we need to use a combination of multiple different models to produce the best result. However, the outcome of sequentially applying multiple models is uncertain and could potentially undermine the quality of the audio substantially. Hence, a more feasible approach is to compare some existing open-source models online, and make modifications accordingly. To make a more fine-grained and thorough extraction, it is better that our teammates first make a classification. For example, when we collect data from video providers such as YouTube, we should label our data points with the type of performance. We decided on a few preliminary categories: studio, concerts, and street

performances. For each category, we need a different model depending on the emphasis. For the videos recorded in studios, the noise is already cancelled by default. So we only need to worry about harmony and instrument removal; For recordings of the concerts, there will be some noise but if the recording is done professionally, the quality will be close to the ones done in studios. However, there might be some unexpected noise in concerts such as cheering and clapping. In such a case, self-supervised learning is the best option; For recordings on streets, it will be more complicated. The street performances are essentially concert performances with poor recording equipment (usually recorded on spectator's phones). Because of that, we need to decide if our focus is on noise cancelling or harmony/instrument cancelling. Our team came up with the conclusion that harmony/instrument removal is more crucial. The reason is that an important metric an auto-tune algorithm looks for is the fundamental frequency. The remaining harmony/instrument will severely undermine the algorithm's accuracy in determining this value, causing false results.

To examine the hypothesis we made above, a benchmark study on the extraction technique across different recording environments will be conducted. This will be an empirical experiment which means the conclusions will come from sensational observations instead of quantitative data. We will inspect the waveform produced by each of the extraction tools and listen to the audio files. At the end, we will choose the best models for our project.

3.2 Visualization:

After successfully extracting the target audio. We need to process this audio and produce different types of plots for better visualization. This module will take the output from the previous Extraction module. At a high level, the visualization module will break down into 3 phases (fig. 2): data preprocessing, data preparation and plotting. Now, let us dive deep into each phase.

Preprocess Phase. Although the input audio files are already free of background noises, there is still some more sanitizing work to do. First of all, we need to normalize the amplitude. Different recordings might have different loudness levels, which brings extra complexities in future analysis. The algorithm we apply to create a consistent waveform amplitude is called peak normalization. The formula is given below:

$$y_{norm} = \frac{y}{\max(|y|)}$$

This ensures that waveform and spectrogram visualizations are not biased by volume differences. We also need to do a step called voice activity detection (VAD). During a performance, it is very common for one to pause at some point and create a temporary silence. These gaps do not materialistically alter the result. However, it will significantly undermine the observer's experience. Hence, this step will be necessary to enhance the clarity of the waveforms.

After sanitizing the audio file, we need to generate the data that will be later used for plotting the diagrams. We name this phase the Data Preparation Phase. We will generate three different datasets as the source for plotting:

1. Waveform data: A waveform representation of the vocal track is computed to show amplitude variations over time. Sudden jumps or flattened sections may indicate manual volume adjustments or dynamic compression associated with pitch correction.
2. Fundamental Frequency (F0) Tracking: The fundamental frequency (F0) represents the perceived pitch of a vocal signal. We use an autocorrelation-based pitch detection algorithm to extract F0 over time:

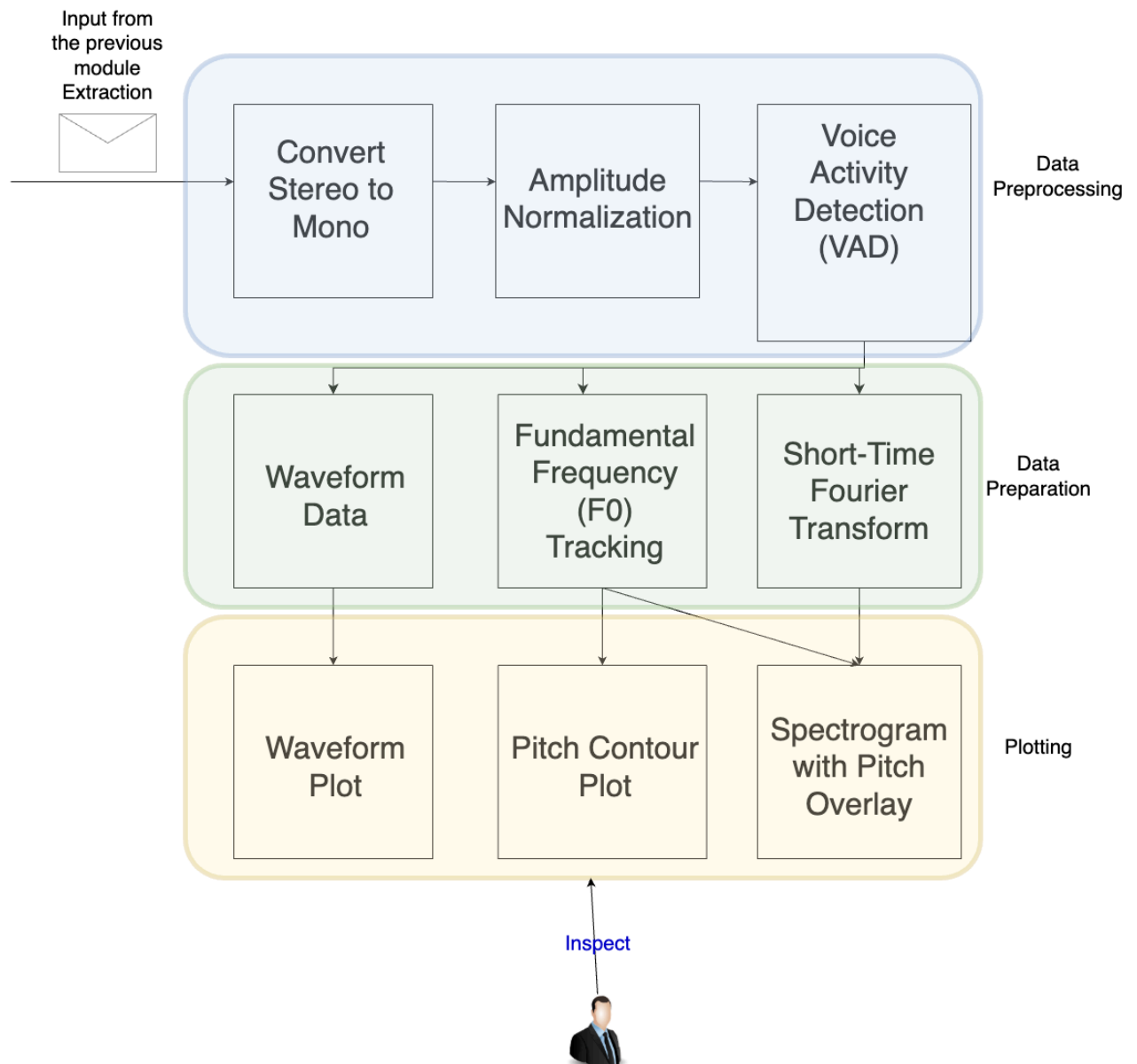
$$r(\tau) = \sum_n x[n]x[n + \tau]$$

The pitch contour derived from F0 tracking helps analysts identify unnatural patterns on the waveform.

3. Short-Time Fourier Transform: This is also another important diagram. To analyze how pitch correction affects frequency content, we compute a spectrogram using STFT. The formula is given below:

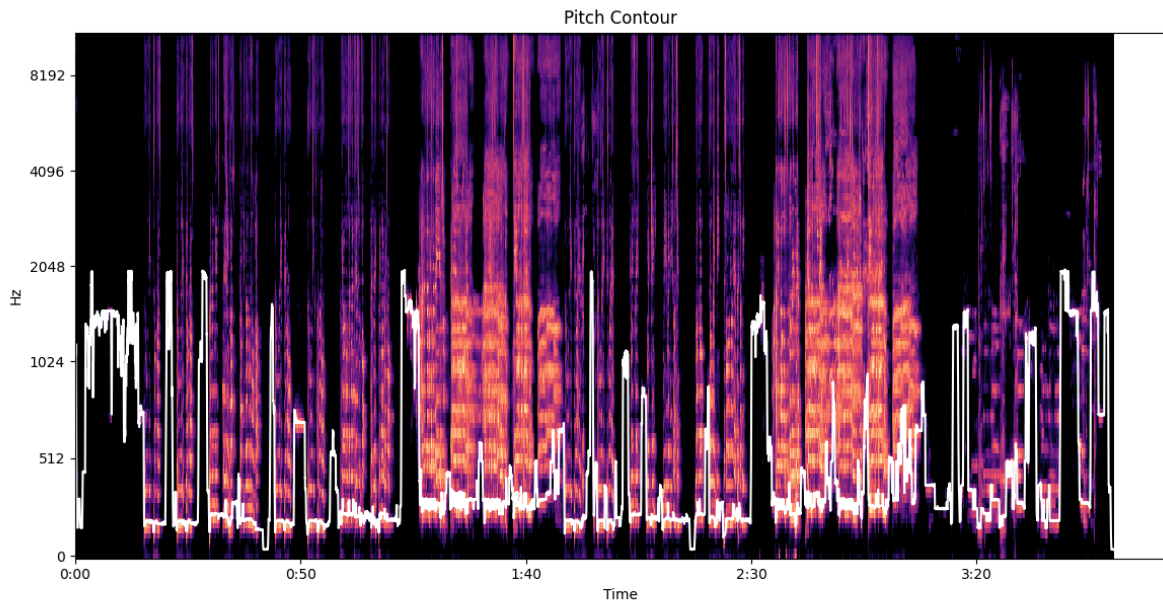
$$X(m, k) = \sum_{n=0}^{N-1} x(n)\omega(n)e^{-j2\pi kn/N}$$

This spectrogram allows visualization of harmonic structures and we can thereby determine whether certain frequencies have been unnaturally adjusted.



(Fig. 2)

These three datasets provide us with all the materials necessary for the plotting phase. At the moment, we are planning to generate at least 3 plots. Waveform plots will display natural loudness variations. The x-axis is time in seconds and the y-axis is amplitude. This plot helps identify unnatural gain smoothing. The pitch contour plot displays how fundamental frequency (F0) evolves over time. The x-axis is time in seconds, and the y-axis is frequency in hertz. This plot can help detect abrupt pitch changes or unnaturally straight pitch lines. The third plot we will generate is a spectrogram with pitch overlay. This plot visualizes the energy of fundamental frequency and harmonics over time. What is worth noticing is that this spectrogram uses data from both “F0 tracking” and “Short-time Fourier Transform”.



3.3 Analysis:

Strictly speaking, we should demonstrate approaches for both streamlined and manual analysis of the plots. However, our team decided to merge these two modules into one because the underlying logic is identical. Since the previous module provides us with 3 plots, we can have a comprehensive and multi-angled understanding of the audio. For each of these 3 plots, there are different indicators that could be potential evidence of a corrected audio.

Waveform plot. This plot is not the crucial plot for the analysis. In other words, the waveform discrepancies cannot be used as definitive evidence for pitch correction. Instead, it will be used as supporting evidence to reinforce our conclusion. The reason is that only some researchers have briefly mentioned the potential change in amplitude in some pitch-correction implementations. The amplitude change in the process of pitch-correction happens at the “fit in” stage to enhance the coherence after the sound has been altered. Hence, if some other plots have shown discrepancies, we will study the waveform plot and investigate the pattern in the same timestamp.

A pitch contour plot is a crucial plot that we need to place an emphasis on. This plot keeps track of the fundamental frequency changes. There are a few indicators we can find on this plot to tell if the original audio has been altered. For example, straightened pitch lines are indicators because natural singing has vibrato and pitch variations. We may also observe stepwise jumps. When the singer’s pitch is too off-tune, the pitch-correction software might fail to adjust the pitch gracefully and result in a very mechanical step-like jump to force the pitch back on the tune.

Spectrograms with pitch overlay also give some important insights. Typically, natural vocals have a slight frequency drift, while auto-tuned harmonics remain perfectly aligned. We need to pay extra attention to the parts that are too perfect. We also need to pay

attention to energy smoothing in formants as tuning algorithms may unnaturally smooth out formant variations.

Everything mentioned above is what we need to understand to observe the pattern with human eyes. However, the stretch goal of this project is to develop our own detection algorithm. Since everything in this project so far is heavily built on theoretical research, speculations, and assumptions, it will be rather difficult to come up with a detailed plan for the detection algorithm. The actual results and plots might be very different from what we expect. However, we can still propose a high-level design and implementation plan. Originally, we planned to use a machine learning classifier that leverages supervised learning. However, due to the scope and time limit of this project, we do not see that as a viable and feasible plan. Rather, we believe that this project can be done perfectly with simply some heuristics. For instance, if the vibrato variation is below a threshold, flag that as possible pitch correction; if stepwise pitch transitions occur in >80% of the vocal segments, flag that as tuned. These heuristics will be relatively easy to implement at a technical level. However, the threshold and metrics will need time. We need to continuously calibrate and reduce to accurate numbers.

4. Evaluation:

Section 3 provided us with a high-level blueprint and theoretical grounding. However, we did not mention any implementation details. More importantly, there are many hypotheses we made in Section 3 without conducting any experiments. In this section, we will mainly focus on how we want to implement, measure, evaluate, and benchmark the methods provided in Section 3. Section 4 is an extension to 3.1 Extraction, 3.2 Visualization, and 3.3 Analysis.

4.1 Extraction

Previously, we researched and proposed numerous models we can use to help the sound extraction such as Spleeter, Demucs, Open-Unmix. However, we have not evaluated the performance of any of them. We are planning to find the best model for each genre. Therefore, we will conduct a comparative empirical study. There are a few objective metrics we can consider to take into account when evaluating the quality of the sound. These metrics are the Signal-to-Distortion Ratio, Signal-to-Interference Ratio, and Perceptual Evaluation of Speech Quality. Additionally, subjective listening tests are also important. This will involve a group-wide assessment. During the evaluation, we need to pay critical attention to how well the harmony and instruments are removed because this will have a substantial effect on the later modules.

4.2 Visualization

The evaluation for visualization should be based purely on observations. We need to make sure the diagrams are readable and interpretable. Most of the work will be based on common sense, for example, does the pitch contour plot match the singer's expected pitch

range? Are the unusual patterns trivial enough for us to recognize? Our team has decided to use Python as the main programming language for this project. As such, we need to compare different Python visualization libraries horizontally.

4.3 Analysis:

Ultimately, we want to investigate the insights of the detection of pitch correction algorithms. We will not only develop the skills to capture the discrepancies using human eyes but also aim to complete our own algorithm. Hence, in our evaluation, we need to compare how well our machine can perform compared to human eyes. This evaluation should be comprehensive and follow the statistical standards. An important thing to note is that we should complete the quality testing for previous modules before this step because we need to control the variable so that we can definitively attribute the factors that cause the performance difference.

Firstly, we need to choose the datasets, since we need to control the variables the datasets should be transparent to the experimenter. In other words, the experimenter should have knowledge of whether pitch correction is used. Therefore, we decided to let one of our team members record some songs with different severity of pitch correction. Also, we need to use different pitch correction softwares to ensure the comprehensiveness of this research. Our preliminary dataset will include raw vocals, mildly corrected vocals, and heavily corrected vocals. Since we are planning to generate these datasets ourselves, we will try to simulate the street and live performance environment respectively. Our detection algorithm will be executed with these datasets in parallel with human-eye detection. Obviously, the samples will be presented to the individuals in a label-agnostic manner.

After the collection of the data, we need to count True Positive, False Positive, True Negative, and False Negative. With those, we can calculate the following:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1\ score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

At the end of the experiment, we want to use the above metrics to determine whether humans or our software perform better. We also need to do some qualitative analysis and continuously improve our model. For example, we should pay additional attention to the timestamps, either 1) the software reports a discrepancy but the human does not, or 2) the human reports a discrepancy but the software does not. We should study the reasons behind the inconsistencies between the software and the human to further polish the algorithm.

5. Mass Production & sociocultural Inference

After a complete test and polish, we would expect a streamlined, integrated module that is capable of processing massive and diverse data. So the next step is to collect as much real data as we can find. Ultimately, we want to generate meaningful insights regarding the prevalence of pitch correction in the music industry.

There are a few insights that our group is interested in. We unanimously share a bias that old music is more “authentic” than modern music. This hypothesis can be validated through our analysis. We need to collect data from different timestamps but control all other variables. We decide to do an analysis on both Western music and Eastern music and compare the industry trend over time in both cultures. We can also visualize the data through a line chart.

Due to the heavy commercialization of pop music, we speculate that the pitch-correction is more extensively used in pop music compared to other genres. Hence, we decide to control other variables but manipulate the genre. We will calculate the average use of pitch correction per genre and visualize it with bar charts. We will see if there exists a higher rate of pitch correction in pop music and if the difference is statistically significant.

There are also some philosophical and ethical debates elicited from research. With the unidirectional and inevitable advancement of technology, we could expect the music correction software to be more and more fine-tuned to the point where the average person can perform any song someday in the future. But is this ethical? If not, how much correction is considered as the barrier to separate ethical and unethical? What will the audience’s reaction be if they know the song is partially created by computers?

6. Conclusion

In this proposal, we demonstrated our blueprint and goals for the project. In general, we have two major tasks. First, we need to develop our own strategies to differentiate pitch-corrected songs from authentic songs. This process can be done manually but preferably systematically. The second part of this project is to analyze the socio-cultural impact on the music industry from the emergence of new pitch-correction technology. We want to see how much pitch correction is used across cultures, time, and genres. We are validating the idea that the commercialization of music and the advancement of pitch correction are mutually reinforcing each other. At the end, we initialize a provocative philosophical debate on whether pitch-correction software is ethical or not.

References

- Gohari, Mahyar, et al. "Spectrogram-Based Detection of Auto-Tuned Vocals in Music Recordings." *arXiv*, 8 Mar. 2024, <https://arxiv.org/abs/2403.05380>.
- Fernando, Rashin, et al. "Hybrid Y-Net Architecture for Singing Voice Separation." *arXiv*, 6 Mar. 2023, <https://arxiv.org/abs/2303.02599>.
- Hennequin, Romain, et al. "Spleeter: A Fast and Efficient Music Source Separation Tool with Pre-trained Deep Learning Models." *arXiv*, 8 Nov. 2019, <https://arxiv.org/abs/1911.13254>.
- Silero Team. (2024). Silero VAD: pre-trained enterprise-grade Voice Activity Detector (VAD), Number Detector and Language Classifier [Computer software]
- CREPE: A Convolutional Representation for Pitch Estimation
Jong Wook Kim, Justin Salamon, Peter Li, Juan Pablo Bello.
Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP), 2018.